

siox

A hardware description language for the whole circuit

Purpose and design · October 2026

Abstract. siox (“silicon oxide”) is a hardware description language and compiler. It describes digital circuits with the precise semantics of VHDL and the structure of Rust: entities and processes, nine-value IEEE logic and delta cycles, written with modules, traits, generics and one strict type system. Designs compile through LLVM to native simulators that run their own tests and write VCD or FST waveforms.

This document explains what siox is for and what it will do. Part I gives the purpose and the principles. Part II describes the digital language as it exists today. Part III describes where it is going: the language features already designed, analogue and mixed-signal modelling, and the design and synthesis layer that turns models into hardware. Each feature is marked with where it stands: implemented, partly implemented, proposal, Phase 2 or Phase 3.

Contents

Part I · Purpose	4
1 Why another HDL	5
1.1 What “the whole circuit” means	5
1.2 Who siox is for	5
2 Principles	6
2.1 Hardware semantics first	6
2.2 Two audiences, one language	6
2.3 Three sigils, one job each	6
2.4 The library over the compiler	6
2.5 Tooling that behaves like a modern compiler	7
2.6 Deterministic	7
2.7 Phased, and honest about it	7
3 A first look	8
Part II · The digital language	10
4 Program structure	11
4.1 Modules and imports	11
4.2 Entities and implementations	11
4.3 Instances and connections	12
4.4 Buses and views	12
4.5 Parameters and generation	13
5 Behaviour	14
5.1 Wires and processes	14
5.2 One assignment operator	14
5.3 Events, clocks and reset	14
5.4 Conditions	15
5.5 Pattern matching	15
5.6 Multiple drivers and resolution	15
5.7 Time	15
6 Types	17
6.1 The kernel	17
6.2 Logic values	17
6.3 Numeric vectors	17
6.4 Ranged integers	17
6.5 Aggregates	18
6.6 Newtypes and aliases	18
6.7 Every value starts defined	18
7 Abstraction	19
7.1 Traits and generic implementations	19
7.2 Operators	19
7.3 Literal prefixes and suffixes	20
7.4 Conversions and indexing	20
7.5 Functions	20
8 Metadata and directives	21
8.1 Declared attributes	21
8.2 Directives	21
8.3 Lints	21
9 Verification	23
9.1 Testbenches are entities	23
9.2 The test executable	23
9.3 Checking and reporting	23

9.4	Stimulus services	24
9.5	Waveforms and debugging	24
10	The compiler	25
10.1	One pipeline	25
10.2	Native simulation	25
10.3	Diagnostics	25
10.4	C functions	25
10.5	Embedding and editors	26
Part III	Where siox is going	27
11	The digital language, growing	28
11.1	Pipelined functions	28
11.2	Macros	28
11.3	<code>core</code> and <code>std</code>	29
11.4	The standard library	30
11.5	Entity methods	30
11.6	<code>derive</code> and the rest of the type system	31
11.7	Compiler foundations	31
11.8	The simulator interface	31
12	Phase 2: analogue and mixed signal	32
12.1	What it will do	32
12.2	What it will not do	32
12.3	How it will be judged	32
13	Phase 3: design and synthesis	33
13.1	Projects and libraries	33
13.2	Foreign HDL, side by side	33
13.3	Constraints and physical intent	33
13.4	Synthesis through a vendor-neutral RTL	33
13.5	Schematics and boards	34
13.6	How it will be judged	34
14	Tools, boundaries and status	35
14.1	The tools around the compiler	35
14.2	What siox will not be	35
14.3	How siox compares	35
14.4	Where things stand	35
14.5	Summary	36
Appendix		37
15	Reference	38
15.1	Keywords	38
15.2	System attributes	38
15.3	Directives	38
15.4	Lints	38
15.5	Glossary	39
15.6	Further reading	39

Part I · Purpose

1 Why another HDL

Hardware engineers today choose between languages that each solve part of the problem.

- **VHDL** has the most careful semantics of the mainstream HDLs. Its nine-value logic, resolved signals and delta-cycle model describe real circuits precisely. Its syntax and type system date from the 1980s, and writing reusable, well-structured code in it is laborious: generics are limited, there are no traits or modules in the modern sense, and the tooling around it is old.
- **Verilog and SystemVerilog** are concise and universally supported, but their semantics are loose: four-value logic, implicit widening and truncation, scheduling races between blocks, and a well-known gap between what simulates and what synthesizes. SystemVerilog added a large verification language on top rather than tightening the design language underneath.
- **Embedded generators** such as Chisel and Amaranth use a general-purpose language to build circuits. They gain that language's abstractions, but the circuit is a data structure the host program constructs, so errors, types and tooling belong to the host, not to the hardware, and simulation is a separate world from generation.
- **Newer HDLs** such as Spade and Clash bring modern language design to hardware. siox shares their direction and borrows from them where they have already solved a problem; Spade's pipelines (§ 11.1) are one example.

None of them covers more than the digital part of a circuit. A real board mixes digital logic with analogue parts, power, clocks and the physical connections between them, and today each of those is described in a different tool, in a different language, by a different team.

siox aims to be one language for the whole circuit: precise enough to simulate digital logic exactly, extensible to analogue behaviour, and structured enough to compose both into a design that existing tools can build.

1.1 What “the whole circuit” means

The project is organised around one rule:

Digital and analogue define components. Design connects and realizes them.

- **Components** are described in the language: a counter, a UART, a PLL model, an amplifier stage, a power rail. Digital components exist today; analogue components are the second phase.
- **Design** is the act of connecting components into a product and handing it to the tools that build it: synthesis for FPGAs and ASICs, netlists and schematics for boards, constraints and pin assignments. That is the third phase.

Each phase builds on the one before without changing what it means. The digital language does not become approximate when analogue arrives, and the design layer does not reinterpret what the components say.

1.2 Who siox is for

- **Hardware engineers** who know VHDL or Verilog and want a language with modern structure, strict types and good tools, without giving up the hardware model they already think in.
- **Software engineers** moving into hardware, who know Rust's style and need a language whose rules catch the mistakes hardware punishes: width mismatches, multiple drivers, latches, uninitialised values.
- **Teams** that build boards and systems, not just chips, and today keep the digital design, the analogue models and the board in separate worlds.

2 Principles

2.1 Hardware semantics first

siox is strict where hardware is strict.

- Every declaration states its type. `let x = e;` without a type is an error.
- Assignment and connection widths must match exactly, and arithmetic does not widen silently: `unsigned[8] + unsigned[8]` wraps at eight bits. Widening and narrowing are written as conversions.
- A signal has one driver unless its type says how parallel drivers combine. Two drivers of an unresolved signal are an error that points at both.
- The logic type is the full IEEE 1076-2019 nine-value `std_ulogic` set (U X 0 1 Z W L H -), and uninitialized and conflicting values propagate through vectors and arithmetic as the standard defines.
- Every signal starts at a defined value: its type's default. A `Logic` starts at 'U', so an undriven signal reads as uninitialized rather than as a plausible '0'.

Behaviour is checked against an independent reference simulator (nvc for VHDL semantics), not against siox's own output.

2.2 Two audiences, one language

siox is written to be familiar to two groups at once.

- From **Rust** it takes modules and `use` imports, `pub` visibility, structs and enums, traits with generic implementations, `impl` blocks, expression syntax including `if` and `match` as expressions, newtypes, and a compiler whose diagnostics and lint levels work like rustc's.
- From **VHDL** it takes the hardware model: entities with directional ports, concurrent processes with sequential bodies, resolved signals, labels on processes and generate blocks, and attributes read with the tick (`clk'event`, `x'length`).

Where the two disagree, siox follows Rust for program structure and VHDL for circuit meaning. When siox's intended behaviour is unclear, the first question is what rustc does, and the answer stops where hardware starts.

2.3 Three sigils, one job each

The punctuation of the language has fixed meanings:

Sigil	Meaning	Example
.	values: fields and method calls	<code>pkt.data, clk.rising()</code>
::	types and modules: paths, variants, associated items	<code>std::logic::Bit, State::Idle</code>
'	attributes: properties of a declaration or signal	<code>sig'event, x'old, probe'keep</code>

A reader can tell from the punctuation alone whether an expression reads a value, names a type, or asks a question about a declaration.

2.4 The library over the compiler

Behaviour belongs in source code, not in the compiler. The logic values, their truth tables, numeric vectors and their operators, time units, encodings and conversions are declared in the standard library as ordinary types and trait implementations. The compiler provides the mechanisms (type checking, operator dispatch, elaboration, code generation) and evaluates the library's definitions; it does not carry a second copy of what and means for `Logic`. A user can define new value types, operators and literal suffixes the same way the standard library does.

The rule cuts the other way too: what only the compiler can do belongs to the compiler, and is reached through the language in a small, fixed set of places: directives, macros, and the hooks the standard library implements (§ 11.3).

2.5 Tooling that behaves like a modern compiler

sioxc is one compiler invocation per file, like `rustc`. It never runs what it builds. A design compiles to a native object; a file with `#[test]` entities compiles to a native test executable that runs its testbenches, filters them by name and writes waveforms. Every diagnostic carries a stable code, warnings are lints with levels set by `#[allow]`/`#[deny]` or on the command line, and the same compiler library powers an editor language server.

2.6 Deterministic

The same source always produces the same IR, the same object and the same simulation, on every machine and every run. Simulation order inside a delta cycle never depends on hash order or thread timing, randomization is seeded and reproducible, and the corpus of example programs is checked for byte-identical output between runs.

2.7 Phased, and honest about it

siox grows in phases and never accepts syntax it cannot yet give meaning to. Analogue constructs are reported as errors in the digital compiler, not ignored. Features that are designed but not built live as written proposals until they land, and removed syntax is an error that names its replacement (`process name { ... }` became `name: process { ... }`, `using` became `use` and `type`).

3 A first look

An eight-bit counter with a testbench:

```
module counter;

entity Counter {
    clk: Bit in,
    rst: Logic in,
    count: unsigned[8] out,
}

impl Counter {
    let value: unsigned[8] = 0;

    update: process {
        if clk.rising() {           // runs only on a rising clock edge
            if rst == '1' { value = 0; }
            else { value = value + 1; }
        }
    }

    count = value;                  // a wire: always equal to `value`
}

#[test]
entity CounterTest {}

impl CounterTest {
    let clk: Bit = '0';
    let rst: Logic = '1';
    let count: unsigned[8];
    let dut: Counter = { .clk = clk, .rst = rst, .count = count };

    clock: process {
        clk = not clk after 5ns;    // free-running clock, 10 ns period
    }

    stimulus: process {
        await 10ns;                 // hold reset for one edge
        rst = '0';
        for i in 0..9 { await clk.rising(); }
        assert!(count == 10, "counter should reach 10");
    }
}
```

The entity declares the interface: three ports, each a name, a type and a direction. The `impl` gives the behaviour. Two kinds of logic sit side by side: a concurrent assignment (`count = value;`) is a wire that always equals its expression, and a process is ordered behaviour, here clocked. `update:` is an optional VHDL-style label that names the process in diagnostics and tools.

The testbench is an entity too, marked with the `#[test]` directive. It instantiates the counter the way a struct literal is written, drives a clock with a delayed self-assignment, and steps through time with `await`.

```
$ sioxc --test counter.siox -o counter-tests
$ ./counter-tests -o counter.vcd
```

```
running 1 test
test counter::CounterTest ... ok

test result: ok. 1 passed; 0 failed; 0 filtered out
```

`sioxc` compiles; the executable it produces runs the tests and writes the waveform. The same separation as `rustc --test`: the compiler never runs what it builds.

Part II · The digital language

4 Program structure

4.1 Modules and imports

implemented

A file is one module, named on its first line. Imports follow Rust, with one change: an import is renamed with `=`, not `as`.

```
module bus::uart;

use std::logic::Logic;           // one item
use std::bits::{unsigned, sext}; // several from one module
use Master = bus::axi::Master;   // renamed import (Rust: `as Master`)
use std::{math::{self, PI}, numeric::{Word8 = Byte}}; // nested groups
use bus::axi::*;                 // glob: every public name
use self::State::{Idle, Busy};   // enum variants, from this module
use super::spi::Clock;           // from the parent module
pub use std::logic::{Bit, Logic}; // re-export

type Word = unsigned[32];        // transparent type alias
pub type Byte = integer<0..255>;
type Pair<T> = Packet<T>;        // generic alias
```

A `use` of a project module loads the file it names (`use bus::spi::Master;` reads `bus/spi.siox`), and `std::` paths come from the standard library. Fully qualified paths work everywhere without an import. A small prelude is in scope in every module, so `Bit`, `Logic`, `unsigned`, `signed`, `string`, `time` and the directives need no import.

Items are private to their module by default and exported with `pub`. Visibility follows the owning container, as in Rust: a struct field or an inherent method is private unless marked `pub`, and nothing can be more visible than its owner.

The rest of Rust's import model works the same way. Importing a module binds its name as a prefix (`use std::math;`, then `math::max(a, b)`). A glob is weaker than any explicit name, and two globs that bring the same name are an error only where the name is used. A `use` may open any block, and its names shadow the module's until the block ends. Importing a variant does not import its enum, and an import cannot pass through a type alias. One import binds a name in every namespace it lives in, so the module `std::assert` and the macro `assert!` never collide.

4.2 Entities and implementations

implemented

An entity is a hardware component's interface: its ports, and nothing else. Its behaviour lives in an implementation, as a Rust type's methods live in an `impl`.

```
entity Uart<BAUD: integer> {
    clk: Bit in,
    rx: Bit in,
    tx: Bit out,
    data: unsigned[8] out,
    valid: Bit out,
}

impl<BAUD: integer> Uart<BAUD> {
    let shift: unsigned[8] = 0;
```

```
let count: integer<0..15> = 0;
// processes, concurrent assignments, instances, functions ...
}
```

A port is written like a struct field, `name: Type`, followed by its direction: `in`, `out` or `inout`. A direction is a permission: an implementation may write its `out` ports and only read its `in` ports. Writing an input is an error. `inout` ports are tristate nets whose parallel drivers resolve.

State, behaviour and sub-instances all live in the implementation. An entity body that holds anything but ports is an error: the interface stays readable at a glance, and an implementation can be split across several `impl` blocks.

4.3 Instances and connections

implemented

An instance is declared with `let`, and its ports are connected the way a struct literal is written: by name, or by position.

```
let rx_uart: Uart<BAUD = 115200> = { .clk = clk, .rx = pin, .tx = tx, .data =
d, .valid = v };
let tx_uart: Uart<BAUD = 115200> = { clk, rx2, tx2, d2, v2 }; // by port order
```

Ports can also be wired after the declaration, through the instance:

```
let dut: Counter<W = 8>;
dut.clk = clk; // drive an input
count = dut.count; // read an output
```

A missing required connection, or a connection to a port the entity does not have, is an error naming the port; an output connected to two producers is the conflicting-driver error, pointing at both connection sites.

4.4 Buses and views

implemented

A struct is layout without direction. A **view** gives a struct a protocol role, with a direction for every field, so one struct serves both ends of a bus:

```
struct Stream<T> { valid: Bit, ready: Bit, data: T }

view Source<T> for Stream<T> { valid out, ready in, data out }
view Sink<T>   for Stream<T> { valid in, ready out, data in }

entity Producer { bus: Stream<unsigned[32]> Source }
entity Consumer { bus: Stream<unsigned[32]> Sink }

impl Top {
  let wire: Stream<unsigned[32]>;
  let p: Producer = { .bus = wire };
  let c: Consumer = { .bus = wire };
}
```

The applied type writes the backing struct first and the view second, where a port's direction goes. Views overload by backing struct, so `Source` can be declared for many protocols without a name clash, and methods and trait implementations can be attached to an applied view (`impl Stream<T> Source`).

4.5 Parameters and generation

implemented

Entities, structs, traits and functions take generic parameters, resolved at elaboration: entity `Counter<W: integer>`, instantiated as `Counter<W = 8>`. Parameters can size ports and state, select types, and drive generation.

A `for` loop or an `if` with a constant condition at the top level of an implementation is **structure**: it is unrolled or selected once, at elaboration.

```
impl Chain<N: integer> {
  let w: unsigned[8][0..N];

  stages: for k in 0..(N - 1) {
    let s: Stage = { .i = w[k], .o = w[k + 1] };
  }

  tap: if DEBUG == 1 {
    let probe: Probe = { .i = w[N] };
  } else {
    let probe: Stub = { .i = w[N] };
  }
}
```

Ranges are inclusive and directional, like a declared index range: `0..2` is 0, 1, 2 and `2..0` counts down. A labelled `for` becomes a scope in the elaborated hierarchy, with one child per iteration keyed by the loop value in square brackets (`chain.stages[0].s`); a labelled `if` is one scope filled by whichever branch is taken (`chain.tap.probe`). The same path appears in the instance tree, the IR, the waveform and the debugger. Labels are optional; an unlabelled loop names its instances with their indices (`s_0`, `s_1`).

An entity may only be instantiated where structure is built: at the top of an implementation or inside a structural `for/if`. Instantiating inside a process, a `match` arm or a function is an error, because control flow that runs while simulating cannot create hardware.

5 Behaviour

5.1 Wires and processes

implemented

An implementation describes behaviour two ways.

- A **concurrent assignment** at the top level is a wire: `y = a and b;` means `y` always equals `a and b`. Each one is its own driver.
- A **process** is a block of ordered statements. Processes run concurrently with each other and with every wire; the statements inside one process run in source order.

```
impl Register {
    update: process {
        if clk.rising() {
            q = d;
        }
    }

    changed = q != q'old; // a wire beside the process
}
```

The compiler infers what a process is sensitive to; there is no VHDL-style sensitivity list. A process may carry a VHDL-style label (`update:`), and so may any assignment (`sum: y = a + b;`). Labels are never required and never change behaviour: they name things in diagnostics, the IR and the debugger.

5.2 One assignment operator

implemented

There is one assignment operator, `=`, and its meaning comes from context: an initializer on a `let`, a wire at the top level, an update inside a process, or a port connection.

Inside an event-controlled block, assignments to signals take effect at the end of the step (next-state semantics), so `a = b; b = a;` on a clock edge swaps the two values. Local `let` bindings inside a process update immediately.

Within one process, a later assignment overrides an earlier one under its condition, which allows the default-then-override style:

```
process {
    y = b; // default
    if sel { y = a; } // override
}

z = if x > 200 { 200 } else if x < 10 { 10 } else { x }; // the same select as an expression
```

There is no `?:` operator; `if` is an expression, as in Rust. Source order never gives priority **between** processes or wires: two independent drivers of one signal are either folded by the signal type's resolution, or an error.

5.3 Events, clocks and reset

implemented

Every digital signal has two system attributes: `'event` (it changed in this delta cycle) and `'old` (its value before the change). Clock edges are built from them, in the standard library, as ordinary trait methods:

```
pub trait ClockLike {
    fn rising(self) -> Bool;
    fn falling(self) -> Bool;
    fn edge(self) -> Bool;
}
```

`clk.rising()` is defined in the standard library from `clk'event` and `clk'old`, the way VHDL's `rising_edge` is. There is no special clock type: any single-bit signal is a clock when an edge is taken on it, and a user type becomes a clock by implementing `ClockLike`.

Reset is ordinary logic. A synchronous reset is a test inside the clocked branch; an asynchronous reset is a test before it:

```
process {
    if rst == '1' { q = 0; }
    else if clk.rising() { q = d; }
}
```

5.4 Conditions

implemented

A condition must be a type that says how to be true: it implements the `Boolean` trait. `Bool` and `Bit` do. `Logic` deliberately does not, because a `Logic` can be 'X' or 'Z': write `rst == '1'`, never `if rst`. A user type (a state enum, say) becomes usable as a condition by implementing `Boolean`.

5.5 Pattern matching

implemented

`match` works on enums, characters, numbers, ranges and bit patterns, with or patterns and a wildcard, as a statement or as an expression. A match that does not cover every value, or has an arm that can never be reached, is a lint.

```
match state {
    State::Idle => { if start { state = State::Busy; } }
    State::Busy | State::Wait => { if done { state = State::Idle; } }
}
```

5.6 Multiple drivers and resolution

implemented

A signal driven from two places (two processes, two wires, two instance outputs) is only legal if its type implements `Resolve`, a commutative fold of the drivers' values. `Logic` implements the IEEE `resolved` function, so a tristate bus of `Logic` works; `Bit` and `ULogic` do not, so two drivers of either are an error that names each source.

5.7 Time

implemented

Simulation time is a value: `time` is a nominal integer of femtoseconds, with unit suffixes (`10ns`, `2us`), and frequency a nominal real (`100MHz`).

- `await 10ns`; advances time, `await clk.rising()`; waits for an edge, and `await cond`; waits until a condition holds. `await` is for testbenches.

- `x = v after 5ns;` schedules a delayed write with VHDL's inertial semantics. It is testbench stimulus, not hardware; in a design it is an error.

The scheduler runs delta cycles: within one simulation time, every process whose inputs changed runs, their writes are applied together, and the cycle repeats until nothing changes; then time advances to the next scheduled event.

6 Types

6.1 The kernel

implemented

The compiler knows only a handful of types: `integer` (a 64-bit signed word for arithmetic and parameters), `real` (IEEE binary64), `Char` (a Unicode code point, deliberately not a number), `Bool` (`true/false`, the type of conditions) and `string`, an array of `Char`. Everything else, including the logic values and the numeric vectors hardware is written in, is declared in the standard library.

6.2 Logic values

implemented

The digital scalars form a chain of newtypes, each adding values or behaviour:

Type	Values	Meaning
Bit	'0' '1'	Two-state, like VHDL <code>bit</code> . Internal signals and clocks.
ULogic	'U' 'X' '0' '1' 'Z' 'W' 'L' 'H' '-'	IEEE <code>std_ulogic</code> : nine values, unresolved.
Logic	as ULogic	IEEE <code>std_logic</code> : the same values, plus resolution, so tristate buses work.

The truth tables for `and`, `or`, `not`, `xor`, `nand`, `nor` and `xnor`, and the resolution function, are the IEEE 1076-2019 tables, written in the standard library as operator implementations and checked row for row against a reference simulator.

A vector of logic carries its metavalues: the simulator keeps, beside each value plane, a companion plane for the elements that are not plain '0'/'1', only where such values can actually occur. 'U' and 'X' propagate through logic, arithmetic and comparisons as the standard says, and waveforms show them as x and z.

6.3 Numeric vectors

implemented

`unsigned[N]` and `signed[N]` are vectors of `Logic` with a numeric meaning, like `ieee.numeric_std`. They are library types: nothing in the compiler tracks signedness, and `signed` differs from `unsigned` only by its operator implementations (a sign-aware `Ord`, an arithmetic `>>`, a signed `/`).

- **Widths are strict.** An assignment or connection must match widths exactly. Arithmetic does not widen: `unsigned[8] + unsigned[8]` is `unsigned[8]` and wraps.
- **Conversions are explicit.** `unsigned[16](a)` zero-extends or truncates; sign extension is the library function `sxt`, so a signed value widens as `signed[16](sxt(s))`. A constant that does not fit its target is a compile error.
- **Arbitrary width.** Vectors are not limited to a machine word: storage, shifts (`unsigned[128](1) << 64`), metavalues and printing keep every bit of values wider than 64.
- **Literals.** `x"AB_CD"` (hex), `o"17"` (octal) and `1X10` (one logic value per character) are bit strings; their width is their digit count.

6.4 Ranged integers

implemented

`integer<lo..hi>` is an integer restricted to a range, stored in the fewest bits that hold it (two's complement when the range goes below zero). The standard library names the familiar ones: `Byte = integer<0..255>`,

Natural, Positive, Short, Int, Long. A constant out of range is a compile error; a value that leaves its range while simulating is reported with the signal's path.

6.5 Aggregates

implemented

- **Structs** are named fields, with positional, named and spread-update literals: `{ ..base, .valid = '1' }`.
- **Enums** are finite states. Their variants can be names (`Idle`, `Busy`) or characters, which is how `Bit` and `Logic` themselves are declared.
- **Arrays** have declared, directional index ranges (`Logic[7..0]`, `unsigned[8][0..15]`); indexing and slicing (`data[7..0]`, `data[..4]`) use the declared indices. The system attributes `'length'`, `'left'`, `'right'`, `'high'`, `'low'`, `'ascending'` and `'range'` read an array's shape at elaboration time.

6.6 Newtypes and aliases

implemented

`type Word = unsigned[32];` is a transparent alias: `Word` is `unsigned[32]`. A new nominal type wraps an existing representation: `struct Meter(real);` and `enum Logic(ULogic);` share a representation but are distinct types, with the conversion between them generated automatically. Derivation never adds members or inherits behaviour; bigger types are built by composition.

6.7 Every value starts defined

implemented

A signal without an initializer starts at its type's default value, given by the `New` trait: `'U'` for `Logic`, `'0'` for `Bit`, zero for numbers, the first variant for an enum. siox signals are always initialized; they may be undriven, and the compiler warns when they are.

7 Abstraction

7.1 Traits and generic implementations

implemented

A trait is a compile-time contract: a set of functions a type must provide. There are no vtables and no run-time dispatch; every call resolves when the design is elaborated.

```
trait Source<T> {
    fn send(self, value: T);
    fn can_send(self) -> Bit;
}

impl<T> Source<T> for Stream<T> Source {
    fn send(self, value: T) {
        self.valid = '1';
        self.data = value;
    }
    fn can_send(self) -> Bit { return self.ready; }
}
```

Generic implementations are written with Rust's binder, `impl<W: integer> Counter<W>`. Bounds restrict generic code to types that provide what it uses, and a blanket implementation over arrays lifts an element's behaviour to every array of it:

```
impl<T: Resolve> Resolve for T[] {
    fn resolve(self, rhs: T[]) -> T[] {
        let result: T[] = self;
        for i in self.range { result[i] = self[i].resolve(rhs[i]); }
        return result;
    }
}
```

7.2 Operators

implemented

Operators are traits, as in Rust. Each standard operator has its own: `Add`, `Sub`, `Mul`, `Div`, `Shl`, `Shr`, `And`, `Or`, `Not` and `Neg` (unary `-`), with a method named after it. A type gets `+` by implementing `Add`, and the method can also be called directly, `a.add(b)`:

```
impl Add<Complex, Complex> for Complex {
    fn add(self, rhs: Complex) -> Complex {
        return Complex { .re = self.re + rhs.re, .im = self.im + rhs.im };
    }
}
```

The traits double as bounds: a generic function over any type that can be added says `T: Add<T, T>`.

Comparisons are two traits, as rustc's `PartialEq` and `PartialOrd`: `Eq` gives `==` and `!=`, `Ord` gives `<`, `<=`, `>` and `>=`, and every method returns `Bool`. A type with equality but no order implements `Eq` alone, and an order may leave values unordered, as IEEE floating point does for NaN:

```
impl Eq<Version> for Version {
    fn eq(self, rhs: Version) -> Bool {
```

```

        return self.major == rhs.major and self.minor == rhs.minor;
    }
}

impl Ord<Version> for Version {
    fn lt(self, rhs: Version) -> Bool {
        return self.major < rhs.major or (self.major == rhs.major and self.minor <
rhs.minor);
    }
    fn le(self, rhs: Version) -> Bool { return not rhs.lt(self); }
}

```

`abs`, `rem` and `mod` are functions, as in mathematics, and live in `std::math` with `min`, `max` and the constants: `abs(x)`, `rem(a, m)` (the dividend's sign) and `mod(a, m)` (the divisor's). They are generic, so one `abs` serves integer, signed, the fixed formats and float alike.

New operators are `CustomOperator`, parameterized by their symbol. Any symbol or word that is not reserved by the grammar can be an operator, and its binding power is bound inside its implementation; the standard operators' binding powers are the grammar's and cannot change:

```

impl CustomOperator<"xor", Logic, Logic> for Logic {
    attr precedence = 35;
    fn apply(self, rhs: Logic) -> Logic { ... }
}

```

That is how `xor`, `nand`, `nor` and `xnor` exist: the standard library declares them; the compiler does not know them.

7.3 Literal prefixes and suffixes

implemented

`10ns`, `100MHz` and `5i` are ordinary literals with a suffix, and the suffix is a trait implementation in source: `impl Suffix<"ns", integer> for time` builds a time from the number. Bit-string prefixes work the same way through `Prefix`. A library can add its own units and literal forms.

7.4 Conversions and indexing

implemented

`T(x)` converts explicitly. Kernel numeric conversions are built in and width-driven; conversions between named types go through `impl From<Source>`

for `T`, so `Complex(10)` or `Logic(b)` are library code. Conversions never happen implicitly. Types that are not built-in arrays can still be indexed by implementing `Index` and `IndexAssign`.

7.5 Functions

implemented

A function is a pure expression: it is inlined where it is called, in hardware as combinational logic, and evaluated at compile time when its arguments are constants, so `unsigned[clog2(DEPTH)]` works as a width. Methods take `self`, and a type's **associated functions** are called through its path (`Unicode::code(c)`, `Entity::helper(x)`). C functions are available through `extern "C"` blocks (§ 10.4).

8 Metadata and directives

siox separates two things other HDLs mix: **metadata**, which only tools read, and **directives**, which change what the compiler does.

	Spelling	Removing it changes
metadata	<code>attr keep for probe = true;</code>	only what a downstream tool sees
directive	<code>#[test], #[allow(...)]</code>	what the compiler emits, accepts, or reports

8.1 Declared attributes

implemented

An attribute is declared once, with a type, the targets it may apply to, and a default. It is then bound from outside the declaration it describes, and read back with the tick:

```
pub attr external_clock: Bool for Pll = false; // declaration
pub attr speed: integer for entity, let = 1;

impl Top {
    attr external_clock for p = true;           // binds the instance `p`
    attr speed = 3;                             // binds the entity `Top`

    let p: Pll = { .clk = clk, .locked = 1 };

    count = p'speed; // the binding on p, else Pll's binding, else the default
}
```

Because every attribute has a default, a read always has a value. Bindings are compile-time constants, so a read costs nothing in the circuit. Binding an attribute twice on one declaration is an error; a binding never silently overrides another. Metadata bound on an instance appears on it in the elaborated instance tree (`p: Pll [external_clock = true]`), for netlist and constraint tools to export.

8.2 Directives

implemented

`#[...]` is reserved for directives. Like rustc's built-in attributes they are part of the compiler and declared nowhere: they cannot be imported, renamed or shadowed, and take no value. A module's own `attr test` is metadata, bound with `attr test for X = ...;`, and never makes a test.

- `#[test]` compiles an entity into the test executable.
- `#[allow(lint, ...)]`, `#[warn]`, `#[deny]` and `#[forbid]` set lint levels.
- `#[latched]` and `#[latch]` will make a function a pipeline and mark its stages (§ 11.1) [proposal](#).

Metadata written as `#[...]` is an error whose help gives the `attr` binding that replaces it.

8.3 Lints

implemented

Every warning is a **lint** with a snake_case name, as in rustc: `possible_latch`, `unused_signal`, `undriven_output`, `combinational_loop`, `dead_assignment`, `non_exhaustive_match`, `incomplete_struct_literal`, and the others. A level is set for the item or statement a directive precedes, for a whole module with `#! [...]`, or for everything from the command line with `-A`, `-W`, `-D` and `-F`:

```
module design;
#[deny(warnings)]                // every warning in this module fails the
build

impl Pipeline {
    #[allow(undriven_output)]    // a probe kept for the waveform
    let probe: unsigned[8];
}
```

The innermost level wins; `forbid` cannot be lowered by anything inside it. A denied lint is a real error, and says which directive or flag made it one.

9 Verification

9.1 Testbenches are entities

implemented

A testbench is an entity marked `#[test]`. It has no ports; it instantiates the design under test, drives it from processes, and checks it. Testbench processes may do what hardware cannot: wait for time to pass, write files, draw random numbers, print.

```
#[test]
entity AdderTest {}

impl AdderTest {
    let a: unsigned[8];
    let b: unsigned[8];
    let sum: unsigned[8];
    let dut: Adder = { .a = a, .b = b, .sum = sum };

    stimulus: process {
        seed(42);
        for i in 1..100 {
            a = unsigned[8](randint(0, 255));
            b = unsigned[8](randint(0, 255));
            await 1ns;
            assert!(sum == a + b, "the adder wraps like unsigned[8]");
        }
        print!("checked {} random additions", 100);
    }
}
```

Several testbenches may live in one file. Each is named by its qualified path (`adder::AdderTest`) and runs independently.

9.2 The test executable

implemented

`sioxc --test file.siox -o tests` compiles every `#[test]` entity in the file into one native executable, as `rustc --test` does. The compiler never runs it. The executable:

- runs every test, or those whose name matches a filter (`./tests adder::AdderTest`);
- reports each test as passed or failed, with the source location of a failed assertion, and counts warnings;
- writes waveforms when asked: `./tests -o run.vcd` (text VCD) or `./tests -o run.fst` (compressed FST), or both in one run. Several tests share one monotonic timeline.

9.3 Checking and reporting

implemented

- `assert!(cond, "message")` fails the test when `cond` is false.
- `warn!(cond, "message")` reports and counts, and the test still passes.
- `print!("x = {}", x)` formats a line. Enum and logic values print symbolically (`Idle`, `'Z'`), characters and strings as text, and numbers in full, however wide.
- `stop()` ends the test as passed so far; `finish()` ends the simulation.
- A value that leaves a ranged type, or a file read that fails, is reported with the signal's path or the source location.

The macros capture their source location, which is why they are macros: a failure names the line of the call, even when the call is inside another macro. `error! ("message")` fails unconditionally, like Rust's `panic!`. All four are declared in `core` (§ 11.2).

9.4 Stimulus services

implemented

- **Randomness** is deterministic: `rand()`, `randint(lo, hi)`, `uniform()` and `seed(n)` come from one generator with a fixed default seed, so a run always reproduces.
- **Files:** `read<string>(path)` decodes UTF-8 text, `read<integer>(path)` and `read<unsigned[16]>(path)` read binary fixtures, and `exists(path)` probes for one. In hardware an initializer read happens at compile time, baking a ROM image; in a testbench it happens when the test runs.

9.5 Waveforms and debugging

implemented

Waveforms record every signal with its hierarchy path: instances and labelled generate scopes nest (`top.stages[0].s.o`), enums appear by name, `Logic` appears with `x` and `z`, and structs and arrays flatten to one trace per field or element. The files open in GTKWave or Surfer.

10 The compiler

10.1 One pipeline

implemented

`siox` is one Rust crate: a library of stages and a thin command-line driver. Each stage hands the next a complete product, and each stage keeps going after an error so a single run reports as much as it can.

Stage	What it does
parse	Tokens and a syntax tree for every module the program imports, transitively.
attributes	Attach <code>attr</code> bindings to their targets and fold attribute reads into values; collect lint levels.
resolve	Every name to the declaration it means: modules, imports, visibility, paths.
type-check	Types, widths, directions, trait bounds, operator dispatch, conversions.
elaborate	Parameters substituted and generation unrolled into a concrete instance tree.
lower	A digital IR: signals, combinational drivers, event-controlled updates, and one control-flow form for every process.
codegen	LLVM, then a native object or test executable linked with a fixed runtime.

The digital IR keeps one distinction from VHDL's model at its centre: a **driver** (a wire: `target = expr` under a condition) and an **event block** (`on condition: next(target) = expr`) are different things, and `'event` and `'old` are first-class operations. Hardware processes and testbench processes both converge on one control-flow representation, validated before code generation, so there is one path from source to executable and one meaning.

10.2 Native simulation

implemented

The compiler does not interpret designs. It emits the design as native code and links it with a small runtime, fixed and independent of the design, that schedules processes in delta cycles, advances simulation time, applies delayed writes with inertial semantics, and writes VCD and FST. A simulation is therefore a native program, with no interpreter in the loop.

10.3 Diagnostics

implemented

Every diagnostic has a stable code (E-P014 is a conflicting driver, W-P002 a possible latch), a primary location, labelled secondary locations, notes and a suggested fix. Migration errors say exactly what to write instead:

```
error: `using` was split into `use` and `type`; this is a type alias
--> design.siox:3:1
    = help: write `pub type Word = unsigned[32];` (`type`)
```

Warnings are lints whose levels are set in source or on the command line (§ 8.3). The compiler prefers no error over a wrong one: where it cannot yet decide something soundly, it stays silent.

10.4 C functions

implemented

C libraries connect through extern blocks, the way the standard library reaches the C math library:

```
extern "C" {  
  pub fn sqrt(x: real) -> real;  
  pub fn labs(v: integer) -> integer;  
}
```

`real` crosses as `double`, `integer` as a 64-bit signed word, and packed numeric values up to 64 bits as an unsigned word. Calls work in hardware expressions, clocked updates and testbenches.

10.5 Embedding and editors

implemented

The compiler is a library first. `siox::compiler` takes a request (a file or an unsaved editor buffer, and the artifact wanted) and returns every product it produced: modules, resolution, types, the instance tree, the IR, and the diagnostics. The command-line driver is a thin adapter over it, and the `siox-lsp` language server uses the same library for live diagnostics, go-to-definition, hover, completion and rename, so editor and compiler never disagree.

Inspection outputs show each stage: `--emit tokens`, `ast`, `source` (the canonical printed form, also the migration tool), `tree` (the instance hierarchy), `ir` and `llvm-ir`.

Part III · Where siox is going

11 The digital language, growing

Phase 1 is a working baseline, not a finished language. This chapter describes what is designed and written down as proposals, in roughly the order it is expected to land. Each proposal lives in the compiler repository under `docs/proposals/` until it is built, and is then folded into the language specification.

11.1 Pipelined functions

proposal

A function is combinational: one clock cycle, however deep its arithmetic. A pipelined datapath today is an entity written by hand, with a register per stage per value and the latency known only to its author. siox adopts Spade's answer: stage boundaries are part of the source, the compiler carries values across them, and latency is checked at every use. It spells them with two directives rather than new keywords, because a stage boundary is not new computation but an instruction about when results are kept.

```
#[latched]
fn mac(clk: Bit, a: signed[16], b: signed[16], acc: signed[32]) -> signed[32] {
  #[latch] {                                     // stage 1
    let product: signed[32] = signed[32](sext(a)) * signed[32](sext(b));
  }
  #[latch]                                     // stage 2
  let sum: signed[32] = product + acc;
  return sum;
}

impl Filter {
  #[latched(2)]
  y = mac(clk, x, coefficient, offset);          // mac's result, two cycles later
}
```

- `#[latched]` makes a function a pipeline; its first parameter is the clock.
- `#[latch]` marks one stage: on a block, everything in the block; without braces, the one statement that follows. At the end of a stage every value it computed or carried is registered, and a `let` inside a stage block stays visible after it, one cycle later: a stage is a step in time, not a scope.
- The depth is the number of stages, stated by `#[latched(N)]` and checked against the body. Every call site states the depth it expects too, so changing a pipeline's depth breaks its users at compile time instead of silently shifting their timing by a cycle.
- `#[latch(name = fetch)]` names a stage, and `stage(fetch).x` or `stage(-1).x` reads a value in another stage, for forwarding. Using a value before the stage that computes it is an error that says how many stages early it is.
- `#[latch(enable = cond)]` stalls a stage while `cond` is false, and a stall propagates to every earlier stage. `stage'ready` and `stage'valid` say whether a stage accepts input and whether its contents are real; valid bits start false, because every siox signal starts at its default.

The pipeline registers are ordinary registers in the IR and in the eventual RTL; no backend needs to understand the directives. One naming question is open: in hardware a **latch** is a level-sensitive element, while these stages are edge-triggered registers, so `#[pipelined]/#[stage]` are possible alternative names.

11.2 Macros

partly implemented

Some abstractions generate structure rather than compute values. Macros give siox a hygienic, declarative way to write them, invoked with `!` so they stay visibly distinct from calls:

```
pub macro debug_signal($name: ident, $ty: type) {
    #[allow(unused_signal)]
    let $name: $ty;
}

impl Cpu {
    debug_signal!(probe, unsigned[32]);
}
```

Implemented, following Rust's declarative macros 2.0:

- Parameters bind syntax fragments (`expr`, `ident`, `type`, `path`, `stmt`, `item`, `tokens`), not values, and a macro may have several forms, chosen by the arguments.
- A call expands to an expression, statements, implementation members or module items, depending on where it stands.
- Expansion happens before name resolution, so generated code is checked exactly as if it had been written by hand.
- Hygiene: names a macro declares cannot capture the caller's names, and the body's other names mean what they mean where the macro is declared.
- Macros are items, imported with `use`; `--emit expanded` shows what hardware a macro produced.
- Repetition unrolls at expansion time: a `$xs: expr...` parameter takes the remaining arguments, and for macro `$x` in `$xs` `join` and `{ $x }` repeats over them.
- An assertion that fails inside a macro reports the line of the call.
- The built-in `assert!`, `warn!`, `print!` and the new `fatal error!` are ordinary macros declared in `core`, each over a compiler primitive that only `core` may use:

```
pub macro assert($cond: expr, $rest: expr...) { builtin # assert($cond, $rest) }
pub macro error($rest: expr...) { builtin # assert(false, $rest) }
```

Not proposed: procedural token-stream macros, user-defined `#[...]` attribute macros, and arbitrary code execution at compile time.

11.3 core and std

partly implemented

The library is split along one question: **could an external library have written this?**

Contents

<code>core</code>	What only the compiler can provide, reached through the language: the kernel types (<code>integer</code> , <code>real</code> , <code>Char</code> , <code>Bool</code> , <code>string</code> , <code>Range</code>); the hook traits the compiler calls (<code>Add</code> and the other operators, <code>CustomOperator</code> , <code>Eq</code> , <code>Ord</code> , <code>Prefix</code> , <code>Suffix</code> , <code>Index</code> , <code>Boolean</code> , <code>Resolve</code> , <code>New</code> , <code>From</code> , <code>LogicEncoding</code>); the macros, including a new <code>fatal error!</code> ; and the simulator services (<code>await</code> , <code>stop</code> , <code>finish</code> , <code>files</code> , <code>randomness</code>). Compiled into <code>sioxc</code> , so it always matches the compiler.
<code>std</code>	Everything a user programs with: <code>Bit</code> , <code>ULogic</code> , <code>Logic</code> and their truth tables, <code>unsigned/signed</code> and conversions, ranged integers, <code>Complex</code> and math, text encodings, time and frequency, fixed point, and, to come, floating point, vectors and matrices.

Implemented: `core` is compiled into `sioxc` and laid out like `rustc`'s: `core::primitive` (`Bool`, `string`, and `integer`'s methods), `core::ops`, `core::cmp` (`Eq`, `Ord`, `Ordering`), `core::convert` (`From`), `core::default` (`New`) and `core::macros`; `std` re-exports each module under the same name. The directives are not declared at all: like `rustc`'s, they are built into the compiler. Each `core` declaration tells the compiler its role with a `lang` item, as `rustc`'s `#[lang = "add"]` does:

```
attr lang for Add = "add";
```

The compiler finds its hooks by role and never by path, and only `core` and `std` may bind `lang`, so a user trait named `Boolean` stays an ordinary trait. The built-in macros are `core` declarations too (§ 11.2). Still to come: the new `std` content.

`std` also gains vendor-neutral spellings for the metadata every synthesis flow wants, mapped by each backend to its vendor's name: `keep`, `async_reg`, `ram_style`, `rom_style`, `fsm_encoding`, `max_fanout`, `mark_debug`, clock frequencies, I/O standards and pin assignments.

11.4 The standard library

partly implemented

`std` is the mandatory, vendor-independent base: data types, conversions and small helpers every technology has. Memories, FIFOs, stream adapters and verification components are IP, and belong to vendor packages and libraries. Each piece comes with documentation and a runnable example:

1. **Synchronizers and reset helpers** implemented: `std::sync`'s two-flop synchronizer, reset synchronizer, edge detector and pulse synchronizer.
2. **Fixed point** implemented: `std::fixed`, after VHDL's `fixed_pkg`, with the binary point in the index range. `ufixed[3..-4]` has four integer and four fraction bits; `arithmetic` keeps the format, and conversions from `real` round and saturate.
3. **Floating point** partly implemented: `std::float`, after VHDL's `float_pkg`: `float[8..-23]` is IEEE-754 binary32, with addition, subtraction, multiplication, IEEE comparison (a NaN is unordered) and conversions, rounding to nearest even. It runs in simulation today; hardware use waits for the compiler to share repeated values when it lowers hardware.
4. **Linear algebra**: vectors and matrices over any numeric element.

11.5 Entity methods

partly implemented

An entity's interface is its ports, and a protocol between two entities (offer a value, learn whether it was taken) is today spread over several ports, declared twice and wired at every instantiation. Entity methods let a component expose that protocol once, as a function whose calls **elaborate into** ports, following Bluespec's method-to-port mapping:

```
impl Fifo {
    pub fn write(self, value: unsigned[8]) -> Bool { ... }
}

impl Producer {
    if clk.rising() {
        if f.write(value) { value = value + 11; }
    }
}
```

A call materialises an enable port, one port per argument and one for the result; nothing is hidden, and the generated ports cost exactly what hand-written ones do. Associated functions without `self` are implemented; pure accessors and action methods with arguments come next.

11.6 `derive` and the rest of the type system

proposal

- `#[derive(Ord)]` and `#[derive(Resolve)]` generating the routine implementations, once a second real use exists.
- Compile-time selection stays a **value**, `std::target`, never `#[cfg]`: both branches are always type-checked, and there is one IR.

11.7 Compiler foundations

proposal

Five changes borrowed from rustc's architecture, each independent:

1. **UI tests**: every diagnostic pinned by a snapshot test, with inline `//~ ERROR` annotations and a `--bless` mode.
2. **Diagnostics for tools**: `--explain E-P014`, JSON output, and fix suggestions that a `--fix` mode can apply, making every future syntax migration one command.
3. **One constant evaluator** shared by every stage, replacing several that grew separately.
4. **No name lookups after resolution**: later stages work only with resolved declaration identities, never with spellings.
5. **Lang items**: the standard library marks the declarations the compiler hooks into, instead of the compiler finding them by path.

11.8 The simulator interface

planned

A stable scheduler interface, shaped like the VPI that existing tools speak, opens the native simulator to external drivers. The first target is cocotb, so Python testbenches can drive siox designs; the same interface serves co-simulation and, later, the mixed-signal coordination of Phase 2.

12 Phase 2: analogue and mixed signal

Phase 2

The second phase adds continuous behaviour to the language without weakening the digital one. A board's power supply, filters, sensors, amplifiers and clocks are as much part of the circuit as its logic, and siox aims to describe them in the same language and simulate them together.

12.1 What it will do

- **Domains and quantities.** An analogue model works with continuous quantities in physical domains: voltage and current in the electrical domain, and later others (thermal, mechanical). A domain defines what is measured **across** a connection (a potential, such as voltage) and what flows **through** it (a flow, such as current).
- **Equations, not assignments.** Analogue behaviour is a set of relations that must hold at every instant: a resistor states that the voltage across it is its resistance times the current through it; a capacitor relates current to the rate of change of voltage. Time derivatives and integrals are first-class ('ddt' is already reserved, and rejected by the digital compiler today).
- **Solvers.** The compiler collects the equations of a whole network into a system, and a solver integrates it over time. Solver choice, tolerances and step control are part of the model, and a solver that fails to converge points back at the source equations and domains, not at matrix indices.
- **Bridges.** Digital and analogue meet through explicit conversions with well-defined event rules: sampling an analogue quantity into a digital value, holding a digital value as an analogue level, detecting a threshold crossing as a digital event, and quantizing. There are no implicit conversions between the two worlds.
- **One timeline.** Digital events and analogue integration share simulation time deterministically: a crossing that produces a digital event lands at a well-defined point in the delta-cycle order, the same every run.

Illustrative sketch: the syntax below is not designed yet.

```
entity Resistor<R: real> { p: Electrical inout, n: Electrical inout }

impl<R: real> Resistor<R> {
  let branch = p -> n;           // the path from p to n
  branch.voltage == R * branch.current; // holds at every instant
}

entity Capacitor<C: real> { p: Electrical inout, n: Electrical inout }

impl<C: real> Capacitor<C> {
  let branch = p -> n;
  branch.current == C * branch.voltage' ddt;
}
```

12.2 What it will not do

- **Change the digital language.** Digital semantics never depend on an analogue solver. A purely digital design compiles and simulates exactly as it does in Phase 1.
- **Share an IR.** Analogue models get their own intermediate representation and solver boundary. The digital IR stays exact and event-driven.
- **Accept analogue syntax early.** Until the phase lands, analogue constructs are errors in the digital compiler, so no design silently depends on something that does not work yet.

12.3 How it will be judged

Phase 2 is done when siox can solve small linear and nonlinear networks, co-simulate a digital controller with an analogue plant (a PWM controller driving a filter and a load, say), order bridge events deterministically, and report solver failures in terms of the source.

13 Phase 3: design and synthesis

Phase 3

The third phase turns component models into designs that existing tools can build. Components are what Phases 1 and 2 describe; **design** is connecting them into a product and realising it.

13.1 Projects and libraries

A real design is many files, libraries and vendors' IP. Phase 3 adds a project and package graph: reusable libraries with versions, discovered by name. The language names libraries and entities without saying whether their implementation is siox, VHDL, Verilog or a vendor database; that choice belongs to the project, not to the source. An instance of a vendor PLL and an instance of a siox counter are written the same way.

13.2 Foreign HDL, side by side

Most designs will mix siox with existing VHDL and Verilog: a vendor's IP core, a legacy block, a third-party interface. siox elaborates such a design with the foreign components as black boxes whose interface, metadata and library are known, links them by library metadata, and carries their names, widths, directions and clocks through to the output. Foreign HDL syntax never appears inside siox source, and compiling foreign HDL inside `sioxc` is only on the table if no existing compiled format can supply what is needed.

13.3 Constraints and physical intent

Clocks and their frequencies, I/O standards, pin assignments, placement and the vendor attributes of § 11.3 are declared in the language, attached with `attr` bindings to the ports and instances they describe, checked against their declarations, and exported to each tool in its own format.

13.4 Synthesis through a vendor-neutral RTL

proposal

siox compiles to hardware before it compiles to another HDL. The design is lowered into a vendor-neutral, synthesis-facing RTL representation that describes elaborated hardware directly:

Object	What it records
modules, ports, instances	the elaborated hierarchy, with directions, widths and names
registers	clock, edge, reset kind and polarity, enable, next value
memories	width, depth and read/write ports, kept intact so tools infer block RAM
operations	<code>add</code> , <code>mul</code> , <code>mux</code> , comparisons with known widths and signedness, kept intact so tools map multipliers to DSP blocks
clock and reset domains	explicit, for CDC analysis, constraints and timing
metadata	vendor-neutral attributes such as <code>keep</code> , with vendor-specific ones namespaced
provenance	the source location and the construct (a macro, a pipeline) that produced each node

By the time a design reaches this representation, nothing of the source language is left to interpret: traits are resolved, generics specialised, macros expanded, attributes looked up, and pipelines turned into explicit registers. A backend only serialises:

- **SystemVerilog**, first, as a small, conservative subset that every tool accepts: modules, `always_ff`, `assign`, instances. The output reads like RTL assembly, not like hand-written code.
- **VHDL**, from the same representation, describing equivalent hardware.
- **CIRCT** and **Yosys**, where their own representations are a better fit.

A legalization step adapts the representation to each backend's subset; it changes form, never meaning, and a backend that cannot represent something faithfully says so rather than reinterpreting the design. - - `emit rtl` shows the representation, answering the question every engineer asks of a high-level HDL: what hardware did it actually generate?

Simulation-only constructs (testbench `await`, file reads while running, random numbers, `after` delays) are reported when compiling for synthesis. Whether a construct may appear in hardware is answered per declaration by the target, not by excluding a library.

13.5 Schematics and boards

The last step is the board. Composing components into a netlist or schematic, with graphical tooling where it helps, connects the digital and analogue models of Phases 1 and 2 to the physical product, and keeps diagnostics, waveforms and debugging correlated with the source throughout.

13.6 How it will be judged

Phase 3 is done when a design containing siox and foreign components elaborates, emits a stable vendor-neutral artifact that preserves hierarchy, names, widths, directions, clocks and constraints, and that artifact goes through at least two synthesis ecosystems, with enough metadata coming back to correlate their reports with the source.

14 Tools, boundaries and status

14.1 The tools around the compiler

- **sioxc** is the compiler: one invocation per file, producing an object, a test executable or an inspection output. It is not a package manager or a test runner, and never will be.
- A **project tool** planned, in the role Cargo plays for Rust, will own projects, packages, dependencies and builds, calling **sioxc** underneath. It lives in its own repository.
- **siox-lsp** implemented gives editors live diagnostics, navigation, completion and rename, using the compiler as a library.
- **The test corpus** implemented is a separate repository of runnable siox programs that every compiler change must pass, in both of the compiler's build configurations.
- **Syntax highlighting** implemented for siox code ships as a Sublime syntax definition (used by this document, Sublime Text and **bat**), coloured the way One Dark Pro colours Rust.

14.2 What siox will not be

- A package manager or test runner inside the compiler.
- A digital language whose meaning depends on an analogue solver.
- A home for vendor-specific primitives as language keywords.
- A container for foreign HDL syntax inside siox source.
- A language with conditional compilation that can hide code from the type checker; target selection is a value that both branches are checked against.
- A language that accepts constructs it cannot yet give meaning to.

14.3 How siox compares

	VHDL	SystemVerilog	Chisel	siox
Logic values	nine-value <code>std_logic</code>	four-value	two-value	nine-value IEEE, as library code
Widths	strict	implicit widening and truncation	inferred	strict; no implicit widening
Multiple drivers	resolved types	net resolution	last connection wins	only with a <code>Resolve</code> impl
Abstraction	generics, packages	classes, interfaces	Scala	traits, generics, views, operators as traits
Simulation	external event- driven simulators	external event- driven simulators	through generated Verilog	native code from the compiler
Testing	testbench entities	testbench modules and classes	Scala test frameworks	<code>#[test]</code> entities, test executables
Beyond digital	VHDL-AMS, a separate standard	—	—	analogue (Phase 2) and design (Phase 3) in one language

14.4 Where things stand

Area	Status
Modules, <code>use/type</code> with Rust's import forms, visibility	implemented
Entities, implementations, instances, views, generics	implemented
Structural generation with labelled scopes	implemented

Area	Status
Processes, wires, delta cycles, events, clocks, time	implemented
Nine-value logic, metavalues, numeric vectors, ranged integers	implemented
Traits, operator traits, custom operators, literal affixes	implemented
Declared attributes, directives, lint levels	implemented
Testbenches, test executables, VCD/FST waveforms	implemented
extern "C", compiler API, language server	implemented
Entity methods	partly implemented
Standard-library build-out	partly implemented
User macros	partly implemented
core/std split, lang items	partly implemented
Pipelined functions, derive	proposal
Compiler foundations (UI tests, JSON, lang items)	proposal
Simulator interface and cocotb	planned
Analogue and mixed signal	Phase 2
Projects, foreign HDL, constraints, vendor-neutral RTL, synthesis	Phase 3

14.5 Summary

siox is a bet that hardware deserves a language with VHDL's precision and Rust's structure, compiled by a toolchain as dependable as a modern software compiler, and that the same language can grow from digital logic to the whole circuit. The digital foundation exists and runs today. The analogue and design phases build on it without changing what the digital language means.

Appendix

15 Reference

15.1 Keywords

Items	module use type pub extern entity impl struct view enum trait attr const let fn process macro
Control	if else match for in return
Directions	in out inout
Contextual	await after where self true false not _
Word operators	and or not, and the standard library's xor nand nor xnor
Removed	using (now use/type), wait (now await); both are errors that name the replacement

15.2 System attributes

x'event	x changed in this delta cycle
x'old	x's value before the change
x'length	number of elements
x'left, x'right	the declared bounds, in written order
x'high, x'low	the larger and smaller bound
x'ascending	whether the declared range counts up
x'range	the declared range, for iteration: for i in x'range
x'name	a declared attribute: the bound value, or the default

'ddt is reserved for Phase 2 and is an error today.

15.3 Directives

#[test]	compile this entity into the test executable
#[allow(lints)]	do not report these lints in the next item or statement
#[warn(lints)]	report them as warnings (the default)
#[deny(lints)]	report them as errors
#[forbid(lints)]	as deny, and nothing inside may lower it
#![level(lints)]	any lint level, for the whole module
#[latched(N)]	a pipelined function of depth N proposal
#[latch]	one pipeline stage: a block, or the statement that follows proposal

15.4 Lints

Lint	Code	Reports
possible_latch	W-P002	a signal assigned on some paths of a combinational process only
unused_signal	W-P003	a signal nothing reads
unused_param	W-P004	a parameter nothing uses
unused_import	W-P005	an import nothing uses
unreachable_match_arm	W-P006	a match arm earlier arms already cover
non_exhaustive_match	W-P007	a match that misses values

Lint	Code	Reports
suspicious_logic_compare	W-P008	a comparison of logic values that is likely a mistake
suspicious_reset	W-P009	a reset pattern that is likely a mistake
combinational_loop	W-P010	a signal that feeds itself with no register in the path
undriven_output	W-P011	an output or signal nothing drives
unconnected_input	W-P012	an instance input left unconnected
dead_assignment	W-P014	an assignment a later one always overrides
unimplemented_attr	W-P015	an attribute no tool reads yet
incomplete_struct_literal	W-P016	a struct literal that leaves fields at their defaults
unknown_lints	W-P017	a lint name that does not exist
warnings	—	every lint at once

15.5 Glossary

Delta cycle One step of simulation within a single instant: every process whose inputs changed runs, their writes are applied together, and the step repeats until nothing changes.

Driver A source of a signal's value: a wire, a process that assigns it, or an instance output connected to it.

Elaboration Turning parameterized entities into a concrete tree of instances, with generation unrolled and parameters substituted.

Entity A hardware component's interface: its ports.

Metavalue A logic value other than plain '0'/'1', such as 'U', 'X' or 'Z'.

Process A block of ordered statements that runs concurrently with everything else.

Resolution Folding the values of several drivers of one signal into one, defined by the signal type's `Resolve` implementation.

View A directional role over a struct, giving each field a direction, so one struct serves both ends of a bus.

15.6 Further reading

- The siox compiler and its documentation, including the language specification and the proposals: <https://github.com/Siox-lang/sioxc>
- The siox test corpus: <https://github.com/Siox-lang/siox-tests>
- IEEE 1076-2019, the VHDL standard, for the nine-value logic system.
- Spade, an expression-based HDL with pipelines: <https://spade-lang.org/>
- The Rust Reference and the rustc development guide, for the conventions siox follows in its structure and its compiler.